University Echahid Hama Lakhdar, El-oued
Institute of Exact Sciences
Department of Computer Science
$2^{ed}Master$:
Artificial Intelligence and Data Science
Semester : 3.2025

Lab 7 : Object Detection using Fast R-CNN

Master Level – Computer Vision

Learning Objectives

After completing this lab, students will be able to :

1. Understand the motivation and structure of Fast R-CNN.
2. Explore the implementation of feature extraction, RoI pooling, and multi-task learning.
3. Train and test a Fast R-CNN on a small dataset.
4. Evaluate detection performance and interpret results.

Task 1 — Review the Architecture

```python
from torchvision.models.detection import fasterrcnn_resnet50_fpn
model = fasterrcnn_resnet50_fpn(pretrained=True)
print(model)
```

Questions :

1. Identify the parts of this model corresponding to Fast R-CNN : shared feature extractor, RoI pooling, and heads.
2. Why does Fast R-CNN use the entire image as CNN input instead of per-region CNNs like R-CNN ?
3. Locate the layer that performs RoI pooling (or RoI Align).
4. What are the two outputs of the RoI head ?

Task 2 — Feature Extraction

```python
import torch
from torchvision import models, transforms
from PIL import Image

img = Image.open('samples/dog.jpg')
transform = transforms.Compose([
    transforms.Resize((800, 800)),
    transforms.ToTensor()
])
x = transform(img).unsqueeze(0)

vgg16 = models.vgg16(weights='IMAGENET1K_V1').features
with torch.no_grad():
    feature_map = vgg16(x)

print("Feature map shape:", feature_map.shape)
```

Questions :

1. What is the spatial size of the output feature map for an 800×800 input ?

2. How does the stride of the backbone affect the size of the feature map ?
3. Why is it important that all RoIs are extracted from this shared feature map ?
4. If the input image resolution is doubled, what happens to the feature map shape and memory usage ?

Task 3 — Region Proposals

```python
import cv2
import selectivesearch

img = cv2.imread('samples/dog.jpg')
img_lbl, regions = selectivesearch.selective_search(img, scale=500,
    sigma=0.9, min_size=50)

candidates = []
for r in regions:
    x, y, w, h = r['rect']
    if w*h > 500 and w/h < 3 and h/w < 3:
        candidates.append((x, y, x+w, y+h))

print("Number of proposals:", len(candidates))
```

Questions :

1. What role do region proposals play in Fast R-CNN ?
2. Why does Fast R-CNN use an external region proposal method ?
3. How many proposals do you observe per image ?
4. Which parameters (`scale`, `sigma`, `min_size`) affect the number and quality of proposals ? Experiment and note the effect.

—

Task 4 — RoI Pooling Layer

```python
import torch
import torchvision.ops as ops

feature_map = torch.randn(1, 512, 50, 50)  # [batch, channels, H, W]
rois = torch.tensor([
    [0, 10, 10, 40, 40],
    [0, 5, 5, 30, 30]
], dtype=torch.float)

pooled = ops.roi_pool(feature_map, rois, output_size=(7,7),
    spatial_scale=1/16)
print(pooled.shape)
```

Questions :

1. What is the shape of the pooled output ?
2. Why must all RoIs be converted to a fixed size before feeding into fully connected layers ?
3. How does `spatial_scale` relate to the backbone stride ?
4. Modify `output_size=(14,14)`. What changes in computation and memory ?
5. Replace `roi_pool` with `roi_align`. What differences do you observe ?

—

Task 5 — Multi-Task Loss Function

```python
import torch
import torch.nn.functional as F

cls_target = torch.tensor([1,0,2])
bbox_target = torch.randn(3,4)
cls_pred = torch.randn(3,3)
bbox_pred = torch.randn(3,4)

L_cls = F.cross_entropy(cls_pred, cls_target)
L_loc = F.smooth_l1_loss(bbox_pred, bbox_target)
L_total = L_cls + 1.0*L_loc
print(f"Classification Loss: {L_cls:.4f}, BBox Loss: {L_loc:.4f}, Total
    : {L_total:.4f}")
```

Questions :

1. Write the multi-task loss equation used in Fast R-CNN.
2. Why does the localization loss only apply to foreground RoIs ?
3. What is the effect of the weighting parameter λ ?
4. Replace `smooth_l1_loss` with `mse_loss`. How does it affect training ?

—

Task 6 — Training & Evaluation

Questions :

1. How are positive and negative RoIs defined (IoU thresholds) ?
2. What is the typical ratio of positive to negative RoIs in a mini-batch ?
3. Which parts of the network are fine-tuned during training : backbone, head, or both ?
4. Plot training loss curves after a few epochs. What pattern do you observe ?
5. Evaluate the model on test images. Report mAP.

—

Task 7 — Visualization & Analysis

```python
import matplotlib.pyplot as plt
import cv2

img = cv2.imread('samples/dog.jpg')
detections = [
    {"box":[80,100,400,400],"label":"dog","score":0.93},
    {"box":[200,120,350,350],"label":"cat","score":0.25}
]
for det in detections:
    x1, y1, x2, y2 = det['box']
    color = (0,255,0) if det['score']>0.5 else (0,0,255)
    cv2.rectangle(img,(x1,y1),(x2,y2),color,2)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.show()
```

Questions :

1. What threshold do you use to filter detections by confidence ?
2. Apply Non-Maximum Suppression (NMS). How does it change the results ?
3. Visualize both ground truth and predicted boxes. Identify mistakes.
4. Discuss at least two failure cases and how Faster R-CNN could address them.

Questions

1. Replace RoI Pooling with RoI Align. How does mAP change ?
2. Reduce proposals from 2000 $\rightarrow$ 300 $\rightarrow$ 100. How does detection speed and accuracy change ?
3. Fine-tune backbone vs freezing it. Compare training loss and accuracy.
4. Apply Fast R-CNN on vignette data set